MODULE 20 · DOMAIN 7 - MAINTAIN INFRASTRUCTURE

Logging, Debugging and Troubleshooting

Turning on verbose logging, choosing which half of Terraform to listen to, and reading the error you actually got.

DOMAIN 7 of 8 COURSE WEIGHT 8% MODULE 20 of 27

admin@sthithapragna.com

Module two split Terraform into **core and plugins**. Debugging is mostly working out which half is talking.

The log levels

TF_LOG VALUE	WHAT YOU GET
TRACE	The most verbose. Internal operations and provider API calls
DEBUG	Detailed operations, without the full noise of TRACE
INFO	General progress messages about what Terraform is doing
WARN	Non-critical issues that do not stop execution
ERROR	Only failures that prevent Terraform continuing
JSON	TRACE level or higher, in a parseable JSON encoding

ON THE EXAM

Unset means no logging at all. TRACE is the most verbose, which is the ordering question people get backwards.

TF_LOG and TF_LOG_PATH

- **TF_LOG** sets the level. It is an environment variable, not a command-line flag.
- **TF_LOG_PATH** forces the log to be **appended** to a specific file.
- **Even when TF_LOG_PATH is set, TF_LOG must also be set, or nothing is logged at all.**
- That is the gotcha. Setting only the path produces an empty file and a confusing hour.
- The file is appended to, so a long debugging session accumulates several runs in one file.

> ON THE EXAM
>
> Setting a destination does not enable logging. TF_LOG is the switch; TF_LOG_PATH only redirects the output.

Listening to one half

- **TF_LOG_CORE** enables logging for Terraform itself. **TF_LOG_PROVIDER** for the provider plugins.
- Both take the **same level values** as TF_LOG, and activate only a subset of the logs.
- Available since **Terraform 0.15**.
- A provider API problem: **TF_LOG_PROVIDER=TRACE**. A graph or state problem: **TF_LOG_CORE=TRACE**.
- TRACE across both halves is enormous. Narrowing to one is usually the difference between readable and unreadable.

> ON THE EXAM
>
> This is module two's split made operational. Decide which half you are debugging, then listen to only that half.

Two warnings about verbose logs

- **TRACE logs include provider API calls**, so request and response bodies can contain sensitive values.
- So a TRACE log **should be treated like state or a plan file**: potentially sensitive, and not attached to tickets casually.
- **The JSON encoding is not a stable interface.** HashiCorp says it may change at any time, without warning.
- So do not build tooling that parses it. It exists to support HashiCorp's own tooling.
- Turn logging off again afterwards. An exported variable outlives the problem it was set for.

> ON THE EXAM
>
> Third artefact in this course that quietly holds secrets, after state and plan files. The reasoning is identical each time.

Other TF_ environment variables

VARIABLE	WHAT IT DOES
TF_VAR_name	Sets an input variable. Module eight, and nearly the weakest source
TF_INPUT	Set to false to disable interactive prompts, like -input=false
TF_CLI_ARGS	Adds arguments to every command. TF_CLI_ARGS_init targets one
TF_WORKSPACE	Selects a CLI workspace without running workspace select
TF_DATA_DIR	Moves the .terraform directory somewhere else
TF_IN_AUTOMATION	Softens output that suggests interactive next steps

ON THE EXAM

TF_CLI_ARGS_init is how a pipeline applies -backend-config without editing every command. Module sixteen's pattern.

Four kinds of error

- **Language errors.** Invalid syntax, wrong types, unresolvable references. Caught before anything runs.
- **State errors.** State disagreeing with reality, or with itself. Locks, drift, missing bindings.
- **Core errors.** Terraform's own logic: cycles, unknown values at plan time, graph problems.
- **Provider errors.** The platform refused. Permissions, quotas, validation, rate limits.
- HashiCorp's troubleshooting guidance groups errors this way, and each kind has a different first move.

> ON THE EXAM
>
> Naming the kind first saves time. A permissions failure and a cycle look equally like 'Terraform is broken' until you classify them.

Which tool for which error

THE KIND OF ERROR	REACH FOR
Language	terraform validate, and terraform fmt. No credentials needed
Expression or type confusion	terraform console, to evaluate it directly
State	terraform state list and show, then plan -refresh-only
Core	TF_LOG_CORE=TRACE, and terraform graph for cycles
Provider	TF_LOG_PROVIDER=TRACE, then the provider's own documentation
A crash	The crash log, plus terraform version, in a bug report

ON THE EXAM

Most of these are the read-only commands from modules six and eighteen. Reach for those before turning on TRACE.

When Terraform crashes

- A genuine panic writes a **crash log** alongside the stack trace on screen.
- **A crash is a bug**, in Terraform or in a provider. It is not something you configure around.
- Report it with the **crash log**, your **terraform version** output, and a minimal reproduction.
- **terraform version** prints the CLI version and every installed provider version. Module six.
- **Check the crash log before sharing it**, for the same reason as a TRACE log.

> ON THE EXAM
>
> Most errors are not crashes. A crash means a stack trace and a panic, not a red error message.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
The log file is empty	TF_LOG_PATH was set without TF_LOG
The provider returned something unexpected	TF_LOG_PROVIDER=TRACE
TRACE output is unreadably large	Narrow to one half with CORE or PROVIDER
Check the code without credentials	terraform validate. Module six
A stack trace and a panic	A crash log, and a bug report
Logs were attached to a public ticket	TRACE logs can contain secrets

ON THE EXAM

Logging questions are mechanical. Classify the error first, then pick the narrowest tool that can see it.

> An engineer is asked to capture verbose logs for a support ticket. She exports TF_LOG_PATH pointing at a file in her home directory and runs terraform apply.
>
> The apply completes. The file exists and is completely empty. She repeats it twice with the same result.

What is wrong?

A TF_LOG must also be set. TF_LOG_PATH alone enables nothing

B The path must be relative to the working directory

C Logs are only written when a command fails

D TF_LOG_PATH was deprecated in favour of the -log-path flag

◆ PAUSE AND DECIDE

What is wrong?

A TF_LOG is the switch

Correct, and documented almost word for word: even when TF_LOG_PATH is set, TF_LOG must be set for any logging to be enabled.

B Path must be relative

Absolute paths work fine. The file was created at the path she gave, which proves Terraform read the variable.

C Only on failure

Logging is level-based, not outcome-based. A successful run at TRACE produces a great deal of output.

D Deprecated for a flag

There is no such flag. Terraform's logging is configured entirely through environment variables.

THE SIGNAL

The empty file is the clue. Terraform read the path variable and created the file, then had no logging enabled to write into it.

> An apply fails intermittently on one resource type, with a message from the cloud provider that the team cannot interpret. They set TF_LOG to TRACE and capture a log, which runs to several hundred megabytes and is dominated by Terraform's internal graph output.
>
> They need to see the API request and response for the failing call.

What is the better approach?

A TF_LOG=DEBUG, which shows API calls with less noise

B TF_LOG_PROVIDER=TRACE, which logs only the provider plugins

C TF_LOG_CORE=TRACE, since the provider is invoked by core

D Run apply with -parallelism=1 so the log contains one operation at a time

◆ PAUSE AND DECIDE

What is the better approach?

A DEBUG instead

Reduces volume by lowering detail, which risks losing the request and response bodies they specifically need.

B TF_LOG_PROVIDER=TRACE

Correct. It keeps full detail on the half they care about and removes core's graph output entirely.

C TF_LOG_CORE=TRACE

Exactly the output they are already drowning in. Core is the half that produced the noise, not the API calls.

D Parallelism 1

Makes the log easier to follow and does nothing about its size or which half is talking. It also slows every apply.

THE SIGNAL

Narrow by component before narrowing by level. Lowering the level throws away detail; choosing a half throws away the wrong subject.

> A pull request fails in a pipeline that has no cloud credentials by design. The failure is an error about an argument name that does not exist on a resource type.
>
> An engineer proposes giving the pipeline read-only credentials so it can run a plan and catch these earlier.

What should be advised?

A Approve. Only a plan can detect an invalid argument name

B This is a language error. terraform validate catches it with no credentials

C Set TF_LOG=TRACE in the pipeline to reveal the argument name

D The error is a provider bug and should be reported upstream

◆ PAUSE AND DECIDE

What should be advised?

A Only plan can catch it

False. Module six: validate checks attribute names and value types using local provider schemas, with no remote access.

B validate, no credentials

Correct. An invalid argument name is a language error, caught by validate after init -backend=false. Credentials are unnecessary.

C TRACE logging

Verbose logs describe what Terraform did. The error message already names the invalid argument; more logging adds nothing.

D A provider bug

The provider correctly reports that the argument does not exist. Nothing is malfunctioning except the configuration.

THE SIGNAL

Classifying the error first avoids granting credentials to solve a problem that never needed them.

> To help a vendor investigate, an engineer captures a full TRACE log of an apply that creates a managed database, and attaches it to a ticket in a system readable across the whole company.
>
> A colleague asks whether that was safe.

What is the correct assessment?

A Safe. Terraform redacts sensitive values in logs as it does in plan output

B Not safe. TRACE logs include API request and response bodies, which can contain secrets

C Safe, provided the variables were marked sensitive

D Not safe, but only because the log includes the backend configuration

◆ PAUSE AND DECIDE

What is the correct assessment?

A Terraform redacts logs

Redaction applies to Terraform's own rendered output. TRACE logs record the underlying API traffic, which is a different layer.

B TRACE holds API bodies

Correct. A managed database creation involves a generated password crossing that boundary, and TRACE records the traffic.

C sensitive protects it

The fifth appearance of this misconception. sensitive controls display in Terraform's output; it has never controlled what is written elsewhere.

D Only the backend config

Backend credentials are a real concern (M16), and naming it as the only concern understates a log that also holds resource-level secrets.

THE SIGNAL

State, plan files, and now TRACE logs. Three artefacts, one rule: obscured on screen has never meant absent from the file.

Module 20 recap

- **TRACE, DEBUG, INFO, WARN, ERROR**, plus JSON. TRACE is the most verbose; unset means nothing.
- **TF_LOG_PATH does nothing without TF_LOG.** The path redirects; the level switches on.
- **TF_LOG_CORE and TF_LOG_PROVIDER** narrow by half. Narrow by component before lowering the level.
- **TRACE logs can contain secrets.** Treat them like state and plan files.
- **Classify the error first:** language, state, core or provider. Each has a different first move.
- **validate needs no credentials.** Most language errors never require a plan.

> ON THE EXAM
>
> The JSON log encoding is explicitly not a stable interface. Do not parse it.